SauceDemo QA Assessment: Project Guide

- **Project:** `saucedemo-qa-assessment`
- **Application under test:** https://www.saucedemo.com/
- **Stack:** Playwright 1.63 · JavaScript (CommonJS, JSDoc + `// @ts-check`) · Node.js 22 · GitHub Actions
- **Primary browser:** Google Chrome (stable channel), with Firefox available as an opt-in cross-browser run
- **Last verified:** 7 Oct 2026 on Google Chrome 146.0.7680.164.
  - Core suite: 5/5 pass.
  - Regression suite: 18/18 pass.
  - Known-defect suite: 6/6 fail, which is expected because each test fails until its bug is fixed.
  - Core and regression also pass on Firefox 155.

This guide covers everything in the project: the purpose of each folder and file, how to install and run the automation, how the tests are designed, and how to extend, debug, and publish them. The assessment write-up itself (defects, AI log, QA strategy) is in `SUBMISSION.md`.

---

1. What this project contains

Deliverable	Where	Assessment task
Exploratory testing probes (re-runnable)	`explore/`	Task A
Defect evidence screenshots	`evidence/`	Task B
Automated test suites: core E2E + negatives, regression, known defects (all 18 test cases)	`pages/`, `tests/`, `utils/`, `playwright.config.js`	Task C
CI pipeline	`.github/workflows/playwright.yml`	Task C
Full submission (defects, AI log, strategy, tools)	`SUBMISSION.md`	Tasks A–D
Human-readable test cases (18 cases, steps to reproduce, expected/actual, status)	`docs/TEST_CASES.md` / `.xlsx` / `.pdf`	Tasks A–C
Quick-start for reviewers	`README.md`	Task C
This guide	`docs/PROJECT_GUIDE.*`	—

---

2. Quick start (5 minutes)

```
# 1. Go to the project
cd ~/personal/saucedemo-qa-assessment

# 2. Install dependencies (exact versions from package-lock.json)
npm ci

# 3. Make sure Google Chrome is available, plus ffmpeg (used to record videos of failed tests)
npx playwright install chrome ffmpeg

# 4. Run the core suite (headless, Google Chrome)
npm test

# 5. Open the HTML report
npm run report

# 6. Optional: the other two suites
npm run test:regression   # 18 tests, all pass
npm run test:defects      # 6 tests, all FAIL on purpose (one per known defect case)
```

Expected output of step 4:

```
Running 5 tests using 5 workers
  ✓ [chrome] › checkout.e2e.spec.js › user can buy two products and totals are calculated correctly @smoke
  ✓ [chrome] › negative.spec.js › locked-out user is blocked with a clear message and stays on login
  ✓ [chrome] › negative.spec.js › checkout is blocked when First Name is missing
  ✓ [chrome] › negative.spec.js › checkout is blocked when Last Name is missing
  ✓ [chrome] › negative.spec.js › checkout is blocked when Postal Code is missing
  5 passed (~3s)
```

For a cross-browser run, use `npm run test:all-browsers`. It runs the same 5 tests on Chrome and Firefox, 10 in total, and needs `npx playwright install firefox` once.

---

3. Prerequisites

Requirement	Version	Check with
Node.js	18 or newer (tested on 22.22)	`node -v`
npm	9 or newer	`npm -v`
Google Chrome	stable channel (tested on 146)	`google-chrome --version` (Linux), *chrome://version* (any OS)
Internet access	to `www.saucedemo.com` and `registry.npmjs.org`	open the site in a browser
OS	Linux, macOS or Windows	—

About Chrome vs Chromium: the suite uses Playwright's `channel: 'chrome'`, which launches the **branded Google Chrome** installed on the machine, from its standard install location. It does not use the Chromium build bundled with Playwright. If Chrome isn't installed, `npx playwright install chrome` installs the current stable Chrome. On Linux this uses the system package manager and may ask for `sudo`.

---

4. Folder and file reference

```
saucedemo-qa-assessment/
├── .github/workflows/playwright.yml   CI pipeline
├── docs/                              Test cases (MD, XLSX, PDF), this guide and the submission
├── evidence/                          Screenshots proving each defect
├── explore/                           Exploratory-testing probes (not regression)
├── pages/                             Page Object Model classes
├── tests/                             Core test specs + fixtures + test data
│   ├── data/                          Users, products, customer, tax rate
│   ├── regression/                    Regression suite (site works correctly today)
│   └── defects/                       Known-defect suite (red until each bug is fixed)
├── utils/                             Small shared helpers
├── playwright.config.js               Playwright configuration (Chrome default)
├── package.json / package-lock.json   Dependencies and npm scripts
├── .gitignore                         Files kept out of git
├── README.md                          Short reviewer-facing instructions
└── SUBMISSION.md                      Full assessment submission
```

Generated at runtime (git-ignored): `node_modules/`, plus one results folder and one report folder per suite: `test-results/` and `playwright-report/` (core), `…-regression/`, and `…-defects/`.

4.1 `pages/`: Page Object Model

Purpose: each class wraps one screen of SauceDemo. It exposes *locators* (how to find elements) and *actions* (what a user does). Tests read like user stories and never contain raw selectors, so a UI change is fixed in one place.

File	Screen	Key members
`LoginPage.js`	Login (`/`)	`goto()` , `login(user, pass)` , `error`
`HeaderComponent.js`	Top bar and side menu on every logged-in page	`cartLink` , `cartBadge` , `openCart()` , `logout()` , `resetAppState()`
`InventoryPage.js`	Products (`/inventory.html`)	`expectLoaded()` , `item(name)` , `priceInCents(name)` , `addToCart(name)` , `sortBy('az'|'za'|'lohi'|'hilo')` , `names()` , `pricesInCents()` , `openProduct(name)` , `removeButtons`
`ProductDetailPage.js`	Product detail (`/inventory-item.html?id=…`)	`goto(id)` , `name` , `price` , `priceInCents()` , `addToCartButton`
`CartPage.js`	Cart (`/cart.html`)	`expectLoaded()` , `readItems()` , `checkout()` ; also exports the shared `readLineItems()` and the JSDoc `LineItem` typedef
`CheckoutInfoPage.js`	Checkout: Your Information (`/checkout-step-one.html`)	`goto()` , `expectLoaded()` , `submit({firstName, lastName, postalCode})` , `error`
`CheckoutOverviewPage.js`	Checkout: Overview (`/checkout-step-two.html`)	`expectLoaded()` , `lineItems` , `readItems()` , `readSummary()` → `{subtotalCents, taxCents, totalCents}` , `finish()`
`CheckoutCompletePage.js`	Checkout: Complete (`/checkout-complete.html`)	`expectOrderConfirmed()` , `backHome()`

Conventions used throughout:

- **JavaScript style:** CommonJS (`require` / `module.exports`). Each file starts with `// @ts-check` , and parameters are typed with JSDoc (e.g. `/** @param {import('@playwright/test').Page} page */`). This gives autocomplete and type warnings in VS Code or Cursor with no build step and no TypeScript dependency.
- **Selectors:** `getByTestId(...)` resolves the site's `data-test` attributes (configured in `playwright.config.js`). Buttons use `getByRole('button', { name })` .
- **Product cards** are found by their visible product name (`item('Sauce Labs Backpack')`), never by position.
- **`expectLoaded()`** asserts the URL and page title before anything is read. This prevents reading the *previous* page during navigation, a race condition found during exploration.

4.2 `tests/` : the three automated suites

Purpose: the automated checks for all 18 test cases. They are split into three suites that run separately, so tests for known bugs can never turn the main run red.

Suite	Command	Purpose	Result today
Core	`npm test`	The assessment's end-to-end flow plus negative tests. Must always pass.	✅ 5/5
Regression	`npm run test:regression`	Behaviour that works correctly today. Must always pass; a failure means something broke.	✅ 18/18
Known defects	`npm run test:defects`	Each test asserts the **correct** behaviour for one defect case, so it **fails while the bug exists** and turns green when it is fixed.	❌ 6/6 fail (expected)

File	Suite	What it does
`fixtures.js`	—	Extends Playwright's `test` with ready-made page objects (`loginPage` , `inventoryPage` , `cartPage` , `productDetailPage` , …) and a **`loggedIn`** fixture that logs in as `standard_user` through the UI and waits for the Products page. Tests just request what they need.
`data/testData.js`	—	Single source of test data: users (`standard` , `lockedOut` , `visual`), all six product names, customer details, and `TAX_RATE = 0.08` . Passwords can be overridden with `SAUCE_PASSWORD` .
`checkout.e2e.spec.js`	Core	**Happy-path purchase journey** (TC-11, tag `@smoke`). See section 5.1.
`negative.spec.js`	Core	Locked-out login (TC-01), and checkout blocked for each missing required field (TC-13 to TC-15). See section 5.2.

File	Suite	What it does
`regression/login.spec.js`	Regression	TC-02: four wrong or missing credential cases, plus a check that the password field is masked
`regression/session.spec.js`	Regression	TC-03: five protected URLs show the login form. TC-04: Back after logout.
`regression/catalogue.spec.js`	Regression	TC-06: four sort orders. TC-07: list price matches the detail page. TC-09: no horizontal scroll at 375 px.
`regression/checkout.spec.js`	Regression	TC-12: three-item totals of $95.97, $7.68 and $103.65
`defects/known-defects.spec.js`	Known defects	One test per defect case: `[D1] TC-18`, `[D2] TC-16`, `[D2] TC-17`, `[D3] TC-05`, `[D4] TC-10`, `[D5] TC-08`. See section 5.4.

Every test title starts with its test case ID (and defect ID), so a red test in the report can be traced directly to `docs/TEST_CASES.md`.

4.3 `docs/TEST_CASES.*`: human-readable test cases

Purpose: every test case written out in plain language, so that someone who doesn't read code can run the tests by hand. Each case has an ID, module, title, type, priority, preconditions, test data, numbered steps to reproduce, expected result, actual result, status, and (where relevant) the defect ID, evidence screenshot, and name of the automated test.

Format	Best for
`TEST_CASES.md`	Reading on GitHub (index table with links to each case)
`TEST_CASES.xlsx`	Test management: a *Summary* sheet with pass/fail per module and a filterable *Test Cases* sheet
`TEST_CASES.pdf`	Sharing or printing

The 18 cases are TC-01 to TC-18, and all of them are automated. Each case shows its suite, the name of its automated test, the number of automated tests, and the command that runs it. 12 pass, and the 6 failures correspond to defects D1–D5.

The 18 cases make **29 automated tests**. TC-02 (5 login messages), TC-03 (5 protected pages) and TC-06 (4 sort options) each run one test per variation, and every other case has one test. So the suites contain 5 core tests, 18 regression tests and 6 known-defect tests.

4.4 `utils/`

File	Purpose
`money.js`	`toCents("$29.99") → 2999`. All money comparisons use integer cents, so floating-point rounding (e.g. `0.1 + 0.2`) can never cause false failures.

4.5 `explore/`: exploratory-testing probes (Task A)

Purpose: the scripts used to *investigate* the application, not to guard against regressions. Each probe tests one risk hypothesis (H01–H17) and prints what it observed as JSON. Every claim in the defect report can be re-checked by re-running them.

File	Contents
`explore.config.js`	Separate Playwright config: Google Chrome, single worker, list reporter, only runs `*.probe.js`. Keeps probes out of the regression suite.
`hypotheses.probe.js`	First round, H01–H13: empty-cart checkout, deep-linking, route protection, session cookie, cart persistence, Reset App State, field validation, tax maths, sorting, accessibility, mobile layout, login messages, seeded-account comparison. The original H06 and H07 are marked `test.skip` because they had the 30-second wait bug (C1 in `SUBMISSION.md`) and were replaced by the fixed versions in `followup.probe.js`.
`followup.probe.js`	Second round: corrected versions of the two probes that timed out (H06, H07), plus H14 (invalid product id), H15 (cart carried to another user), H16 (Back after order).
`followup2.probe.js`	Third round: clean D4 evidence with the menu closed, H14b (blank checkout page), H17 (Dynamic Catalog menu).
`*.png`	Scratch screenshots that are not defect evidence (mobile layout, invalid-id page, menu).
`results.json`	Last JSON output of `hypotheses.probe.js` (git-ignored, regenerated on each run).

Run all probes:

```
npm run explore
```

Run one probe file:

```
npx playwright test --config explore/explore.config.js explore/followup.probe.js
```

> Probes intentionally **record** behaviour instead of asserting it, so they "pass" even when they find a bug. Read the printed `### Hxx` JSON blocks for the results. Running the probes regenerates the screenshots in `evidence/`, and each evidence file is written by exactly one probe.

4.6 `evidence/`: defect screenshots (Task B)

Purpose: visual proof for each defect in `SUBMISSION.md`. The file-name prefix is the defect number. All screenshots were captured in Google Chrome 146.

File	Shows
`D1a-empty-cart-overview.png`	Checkout overview with `Total: $0.00` and an empty cart
`D1b-empty-cart-order-complete.png`	"Thank you for your order!" for that empty cart
`D2a-deeplink-overview-no-info.png`	Overview reached by URL without entering customer info
`D2b-whitespace-only-info.png`	Spaces typed into First/Last/Postal
`D2c-whitespace-accepted-overview.png`	Those spaces accepted; Finish available
`D3-cart-carried-to-different-user.png`	Second user sees the first user's Fleece Jacket
`D4-reset-state-stale-remove-buttons.png`	Badge cleared but two "Remove" buttons remain
`D5a-item-not-found-add-to-cart.png`	Placeholder "ITEM NOT FOUND" product added, badge = 1
`D5b-cart-after-invalid-item.png`	Badge 1, cart lists nothing
`D5c-checkout-overview-crash.png`	Checkout overview renders a blank page

4.7 `.github/workflows/`: Continuous Integration

`playwright.yml` runs on every push to `main`, on every pull request, and manually through *Run workflow*. The manual run has an **"Also run on Firefox"** checkbox. Its steps:

1. Check out the code and set up Node 22 with an npm cache.
2. Run `npm ci`.
3. Run `npx playwright install --with-deps chrome ffmpeg`, which installs branded Google Chrome stable and the video recorder on the CI runner.
4. Install Firefox, only if the Firefox checkbox was ticked.
5. Run `npm run lint`, which syntax-checks every JavaScript file.
6. **Core suite** (`npm test`) is **blocking**: a failure fails the build.
7. **Regression suite** (`npm run test:regression`) is **blocking**.
8. **Known-defect suite** (`npm run test:defects`) is **informational**. It has `continue-on-error: true`, so its expected failures never fail the build, and it gets no retries. `CI=true` gives the other suites 1 retry, 2 workers, and `forbidOnly`.
9. Upload all three HTML reports as one artifact, `playwright-reports`, kept for 14 days, even when tests fail.

To see results on GitHub, open the repo → **Actions** → the run → **Artifacts** → download `playwright-reports`, unzip it, and open `index.html` in the folder for the suite you want. A green build with a yellow "known-defect" step means the bugs are still present. When that step turns green, a defect has been fixed.

4.8 Root files

File	Purpose
`playwright.config.js`	Suite selection (`SUITE`), test directory, timeouts, reporters, browser projects (Chrome by default, Firefox when `ALL_BROWSERS=1`), base URL, and the `data-test` test-id attribute (details in section 6).
`package.json`	The only dependency is `@playwright/test`. Also holds the npm scripts (section 7).

File	Purpose
`package-lock.json`	Exact dependency versions so `npm ci` is reproducible.
`.gitignore`	Excludes `node_modules`, reports, test results, `explore/results.json`, and `.env`.
`README.md`	Concise run instructions for reviewers.
`SUBMISSION.md`	The assessment answer: summary, risk ranking, hypotheses, AI prompt log, 5 defects, automation notes, QA strategy, tools, evidence index.

---

5. What the automated tests verify

5.1 `checkout.e2e.spec.js`: purchase journey (`@smoke`)

Step	Assertion
Log in as `standard_user` (fixture)	URL is `/inventory.html` and title is "Products"
Read catalogue prices for *Sauce Labs Backpack* and *Sauce Labs Fleece Jacket*, add both	Each button changes to "Remove"; cart badge = **2**
Open cart	Exactly 2 rows. Names, prices (in cents) and quantity 1 **equal** the catalogue values
Checkout and submit customer info	Lands on "Checkout: Your Information", then on the Overview
Overview	Same 2 items. **Item total = sum of prices**, **tax = round(item total × 8 %)**, **total = item total + tax**
Finish	URL `/checkout-complete.html`, title "Checkout: Complete!", header "Thank you for your order!", badge hidden
Back Home and open cart	No "Remove" buttons on Products; cart has **0** rows

The journey is split into `test.step(...)` blocks, so the HTML report and traces show exactly which stage failed.

5.2 `negative.spec.js`

Test	Assertion
Locked-out user	Error text is exactly `Epic sadface: Sorry, this user has been locked out.`, the URL stays on the login page, and no product list is rendered
Missing First Name	`Error: First Name is required`, stays on step one
Missing Last Name	`Error: Last Name is required`, stays on step one
Missing Postal Code	`Error: Postal Code is required`, stays on step one

5.3 Regression suite (`tests/regression/`)

Test case	Assertion
TC-02 Login validation (4 tests + masked password)	Exact error text for: both empty, password empty, wrong password, wrong-case username. Login form stays visible, and the password field has `type="password"`
TC-03 Protected pages (5 tests)	Each URL shows the login button and `Epic sadface: You can only access '<path>' when you are logged in.`, and no page title
TC-04 Back after logout	Error about `/inventory.html` is shown, and there is no product list
TC-06 Sorting (4 tests)	List order equals a correctly sorted copy. First items are Backpack (A→Z), Red T-Shirt (Z→A), Onesie $7.99 (low→high), and Fleece Jacket $49.99 (high→low)
TC-07 List vs detail price	The detail page shows the same product and the same price ($29.99) as the list
TC-09 Mobile layout	At a 375 × 667 viewport, all 6 cards render and the page width ≤ the viewport width (no horizontal scroll)
TC-12 Three-item totals	Item total 9597 cents, tax 768 cents, total 10365 cents

5.4 Known-defect suite (`tests/defects/known-defects.spec.js`)

Each test asserts what **should** happen. It fails today with a message that describes the defect. Where one case has several expectations, the test uses `expect.soft`, so the report lists every broken part instead of stopping at the first.

Test	Asserts (correct behaviour)	Fails today with
`[D1] TC-18` empty-cart order	Checkout cannot start with an empty cart	URL is `checkout-step-one.html`
`[D2] TC-16` spaces-only details	`Error: First Name is required` is shown	No error element; the overview opens
`[D2] TC-17` skip step by URL	Opening step two redirects to step one	URL stays on `checkout-step-two.html`
`[D3] TC-05` cart carried to another user	Second user sees no badge and an empty cart	Badge visible; cart has 1 item
`[D4] TC-10` Reset App State	Badge hidden **and** 0 "Remove" buttons	Badge passes; 2 "Remove" buttons remain
`[D5] TC-08` invalid product id	No Add to cart button, badge = cart rows, and the overview renders	All three soft checks fail: button present, badge 1 vs 0 rows, blank overview

When a defect is fixed, its test turns green. Move it into `tests/regression/` so it then guards against the bug coming back.

5.5 Proof the tests can fail

To confirm the assertions are meaningful, `TAX_RATE` was temporarily changed to `0.09`. The E2E test failed with:

```
Error: tax = 9% of item total
Expected: 720
Received: 640
```

You can repeat this check with any expected value to see the suite turn red.

---

6. Configuration (`playwright.config.js`)

Setting	Local	CI (`CI=true`)	Why
`baseURL`	`https://www.saucedemo.com` (override with `BASE_URL`)	same	Tests use relative paths like `/cart.html`
`testIdAttribute`	`data-test`	same	Lets `getByTestId` use the site's built-in test hooks
`headless`	`true`	`true`	Runs without opening a window
`testMatch` (via `SUITE`)	`core` unless set	same	`core` = `tests/*.spec.js` ; `regression` = `tests/regression/` ; `defects` = `tests/defects/` ; `all` = all three
`retries`	`0`	`1` (`0` for the defect suite)	Failures stay visible locally; a test that passes on retry is labelled *flaky* in CI, not hidden
`workers`	auto (CPU count)	`2`	Parallel execution
`fullyParallel`	`true`	`true`	Every test runs independently
`timeout` / `expect.timeout`	30 s / 5 s	same	Bounded waits; no fixed sleeps anywhere
`trace` , `video`	kept only on failure	same	Debug artefacts without bloating passing runs
`screenshot`	only on failure	same	—
Projects	`chrome` (Google Chrome, `channel: 'chrome'`); `firefox` added when `ALL_BROWSERS=1`	same (Firefox via workflow checkbox)	Tests run on the branded browser real users have; cross-browser is opt-in

Setting	Local	CI (`CI=true`)	Why
Reporters	`list` (console) + `html`	same	Readable logs and a browsable report. Each suite writes its own report (`playwright-report/` , `playwright-report-regression/` , `playwright-report-defects/`), so one suite never overwrites another

Test isolation: Playwright gives every test a fresh browser context, so cookies and `localStorage` (where SauceDemo keeps the cart) are always clean. No test depends on another or on run order.

Environment variables

Variable	Default	Use
`BASE_URL`	`https://www.saucedemo.com`	Point the suite at another environment
`SAUCE_PASSWORD`	`secret_sauce`	Avoid hard-coding a password in CI
`ALL_BROWSERS`	unset	Set to `1` to run on Firefox as well as Chrome
`SUITE`	`core`	`core` , `regression` , `defects` or `all` . The npm scripts set it for you
`CI`	unset	Set automatically by GitHub Actions; switches to CI settings

Example: `BASE_URL=https://staging.example.com npx playwright test`

---

7. Running the automation: command reference

Goal	Command
Core suite (Google Chrome, headless)	`npm test`
Regression suite	`npm run test:regression`
Known-defect suite (expected to fail)	`npm run test:defects`
All three suites in one run	`npm run test:everything` (exits red because of the defect tests)
One defect only	`npm run test:defects -- -g D3`
Core suite on Chrome + Firefox	`npm run test:all-browsers`
Regression on Chrome + Firefox	`ALL_BROWSERS=1 npm run test:regression`
Firefox only	`ALL_BROWSERS=1 npx playwright test --project firefox`
Only the purchase journey	`npm run test:smoke`
One file	`npx playwright test tests/negative.spec.js`
One test by name	`npx playwright test -g "Postal Code"`
Watch the browser	`npm run test:headed`
Step through interactively (Inspector)	`npx playwright test --debug -g @smoke`
Visual test runner (UI mode)	`npx playwright test --ui`
Check stability (repeat 10×)	`npx playwright test --repeat-each=10`
Syntax-check all JS files	`npm run lint`
Open the last HTML report	`npm run report` (core), `npm run report:regression` , `npm run report:defects`
Open a failure trace	`npx playwright show-trace test-results/<test-folder>/trace.zip`
Run exploratory probes	`npm run explore`

---

8. Reading results and debugging failures

1. **Console:** the `list` reporter prints ✓/✘ per test, and failures show the assertion message (e.g. `item total = sum of line prices`), expected vs received, and the source line.
2. **HTML report** (`npm run report`): filter by browser or status, expand `test.step` sections, and view attached screenshots, videos, and traces for failed tests.
3. **Trace viewer:** a time-travel recording of a failed test, with DOM snapshots, network, and console for every action. It's the fastest way to see *why* a locator didn't match.
4. **Common causes:**

Symptom	Likely cause	Fix
`Chromium distribution 'chrome' is not found`	Google Chrome not installed in its standard location	Install Chrome, or run `npx playwright install chrome`
Every test fails at `browserContext.newPage` with "Video rendering requires ffmpeg binary"	Playwright's ffmpeg is missing (or its browser cache was cleared). It is needed even with branded Chrome because `video: 'retain-on-failure'` records every test	`npx playwright install ffmpeg`
`browserType.launch: Executable doesn't exist` (Firefox)	Firefox not installed for a cross-browser run	`npx playwright install firefox`
Missing shared libraries on Linux	System dependencies missing	`npx playwright install-deps` (needs sudo)
All tests time out on `goto`	No internet access, or saucedemo.com is down	Check the site in a browser and any proxy settings
Text assertion fails after a site update	SauceDemo changed its copy	Update the expected text in the spec or page object
Tax assertion fails	Tax rule changed	Confirm the new rule, then update `TAX_RATE`

9. Extending the suite

Add a new page: create `pages/NewPage.js` following the existing pattern (locators in the constructor, an `expectLoaded()` method, action methods), then register it in `tests/fixtures.js`:

```
// pages/NewPage.js
// @ts-check
const { expect } = require('@playwright/test');

class NewPage {
  /** @param {import('@playwright/test').Page} page */
  constructor(page) {
    this.page = page;
    this.title = page.getByTestId('title');
  }

  async expectLoaded() {
    await expect(this.title).toHaveText('New Page');
  }
}

module.exports = { NewPage };
```

```
// tests/fixtures.js — add inside base.test.extend({...}) and to the Pages typedef
newPage: async ({ page }, use) => use(new NewPage(page)),
```

Rules to keep the suite healthy:

- Use `getByTestId` or `getByRole`; never CSS classes or `nth-child`.
- Never use `page.waitForTimeout()`; use `expect(...)` web-first assertions.
- Call `expectLoaded()` before reading data from a page.
- Compare money in cents with `toCents`.
- Start new files with `// @ts-check` and add JSDoc types to constructor and method parameters.
- Assert *business outcomes* (totals, cart contents), not just visibility.
- Before merging a new test, make it fail once on purpose to prove it can.

Where to put a new test:

- If it should always pass, put it in `tests/regression/` (or the core folder if it is part of the main E2E flow).
- If it covers a newly found bug, put it in `tests/defects/known-defects.spec.js`. Assert the **correct** behaviour, so the test fails until the bug is fixed. Do **not** use `test.fail()`, because that would hide the bug behind a green tick. Start the title with the defect and test case IDs, e.g. `[D6] TC-19 …`.
- When a defect is fixed and its test turns green, move the test into `tests/regression/`.

Add a new regression test (in `tests/regression/`, so the imports go up one level):

```
// @ts-check
const { test, expect } = require('../fixtures');
const { products } = require('../data/testData');

test('TC-19 removing an item updates the badge', async ({ loggedIn: inventory }) => {
  await inventory.addToCart(products.backpack);
  await expect(inventory.header.cartBadge).toHaveText('1');
  await inventory.item(products.backpack).getByRole('button', { name: 'Remove' }).click();
  await expect(inventory.header.cartBadge).toBeHidden();
});
```

---

10. Publishing to your personal GitHub

The project is a **standalone git repository** with no remote and no connection to any work repository. Its local git identity is intentionally blank, so nothing can be committed under a work email by accident.

```
cd ~/personal/saucedemo-qa-assessment

# 1. Set your PERSONAL identity for this repo only
git config user.name  "Your Name"
git config user.email "you@personal-email.com"

# 2. First commit
git add .
git commit -m "SauceDemo QA assessment: exploration, defects, Playwright suite"

# 3. Create an EMPTY public repo on github.com (personal account), then:
git remote add origin https://github.com/<your-username>/saucedemo-qa-assessment.git
git push -u origin main
```

After pushing:

- Replace `<add your public GitHub URL here>` in section 4 of `SUBMISSION.md`, then commit and push again.
- Open the **Actions** tab to confirm the CI run is green.

Check before pushing: `git remote -v` should show only your personal repo, and `git log --format='%an <%ae>'` should show only your personal email.

---

11. Assessment task mapping

Task	Requirement	Where it is answered
A	Risk-based exploration, AI prompts, validated/rejected ideas	`SUBMISSION.md` §2, `explore/`
B	Up to 5 defects with steps, expected/actual, severity, impact, evidence	`SUBMISSION.md` §3, `evidence/`
C	Runnable E2E test, setup instructions, AI contribution, manual fixes, limitations	`tests/`, `pages/`, `README.md`, `SUBMISSION.md` §4, this guide
D	1-page AI-driven QA strategy	`SUBMISSION.md` §5
—	Tools list	`SUBMISSION.md` §6